Producer-Consumer (Semaphores)

EXP 07

AIM:

To write a program to implement a solution to producer consumer problem using semaphores.

ALGORITHM

1. Start
2. Include required header files
3. Define buffer size
4. Declare mutex, empty and full semaphores
5. Create producer function
6. Producer produces an item
7. Wait for empty space using sem_wait(&empty)
8. Lock buffer using sem_wait(&mutex)
9. Add item into buffer
10. Unlock buffer using sem_post(&mutex)
11. Signal full slot using sem_post(&full)
12. Create consumer function
13. Consumer waits for full slot using sem_wait(&full)
14. Lock buffer using sem_wait(&mutex)
15. Remove item from buffer
16. Unlock buffer and signal empty space
17. End

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;
```

```
sem_t empty, full, mutex;

void *producer(void *arg) {
    int item;

    for(int i = 1; i <= 5; i++) {
        item = i;

        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Producer produced item %d\n", item);
        in = (in + 1) % SIZE;

        sem_post(&mutex);
        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg) {
    int item;

    for(int i = 1; i <= 5; i++) {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumer consumed item %d\n", item);
        out = (out + 1) % SIZE;

        sem_post(&mutex);
```

```
        sem_post(&empty);

        sleep(1);
    }

    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}
```

RESULT:

Thus, the Producer-Consumer problem using semaphores was successfully implemented and executed.